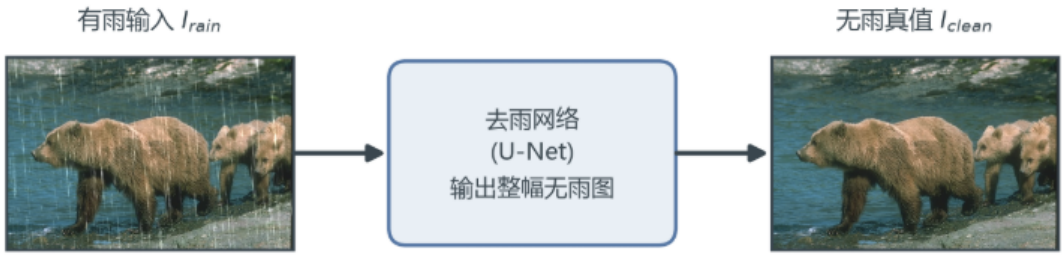
Part 1-任务一

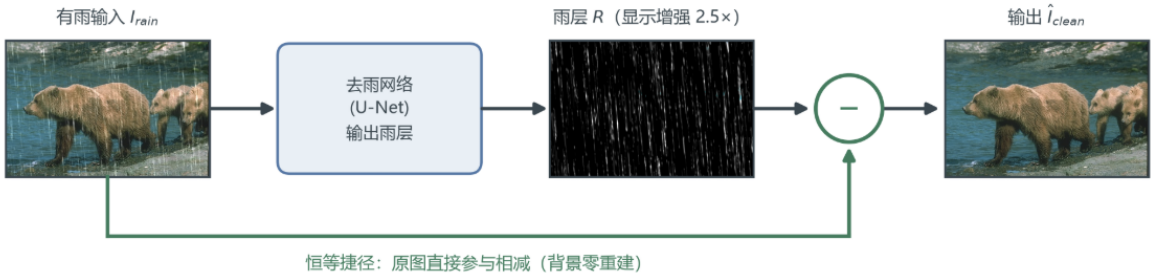
基础问题：

我认为应该学习雨层，而不是直接学习无雨图像。

直接学习：



残差学习：



残差学习的优势是直接对原图进行修改、优化，可以保证生成图像的质量和与原图相似性，也正好符合$ResNet$的思想：$H(x) = F(x) + x$；

直接学习需要对随机生成的图像进行优化和拟合，可能导致生成图像模糊和与原图偏差过大。

并且，训练集中各图像的细节与风格差异巨大，可能导致训练出来的模型欠拟合和过拟合，达不到目的效果。而训练集中各图片的雨滴的细节与风格类似，训练出来的模型拟合能力更强，去雨效果更好。

附加问题：

1：

物理上，雨丝与物体上类似纹路还是有区别。

比如：雨丝细长，密度、方向一致，能够跨过物体延伸（位于图层最上面）；纹理通常跟随物体表面，方向不一致。

模型中，通过大量图片训练并学习了雨丝，而其他物体纹路因样本少且不统一而在模型训练中被忽略。

即训练让模型学到了“什么样的亮线通常是雨出现”。

2：

不能保证完全恢复，只能通过模型推测，可能导致遮挡部分模糊或失真。

Part 1-任务二

假设模型输出图像是 I，真实无雨图是 K。

$d_i = I_i - K_i$

L1 范数：$|d_1| + |d_2| + \ldots + |d_N|$，几何意义表示曼哈顿距离，即一维坐标系中两点距离的总和。

L1 损失：$\dfrac{|d_1| + |d_2| + \ldots + |d_N|}{N}$

L2 范数：$\sqrt{{d_1}^2 + {d_2}^2 + \ldots + {d_N}^2}$，几何意义表示欧式距离，即二维坐标系中两点距离的总和。

L2 损失：$\dfrac{\sqrt{{d_1}^2 + {d_2}^2 + \ldots + {d_N}^2}}{N}$

计算过程：

1：模型输出 I和真实图 K 都转成浮点数

2：逐像素、逐通道相减，得到误差张量 $D = I - K$，再取绝对值或平方

3：把所有 $H \times W \times 3$ 个数加起来，再除以 $H \times W \times 3$，得到一个标量损失，反向传播。

```
L1:  loss = mean(|I - K|)
L2:  loss = mean((I - K)²)
```

SSIM

数值范围：

-1~ 1

1：两张图完全一样

0：局部结构没有相似性

< 0：局部结构相反（比如亮暗颠倒）

考虑角度：

$SSIM$ 不是逐像素比差值，而是比较局部区域的三特征：

亮度：局部平均亮度

对比度：局部明暗变化幅度（用方差/标准差衡量）

结构：局部纹理、边缘细节

评估对象：

$PSNR$：把整张图所有像素的误差汇总，算一个全局 MSE。

$SSIM$：每次只看一个局部窗口（比如 11×11 的小块），用这个小块里的均值、方差、协方差算一个局部 $SSIM$，然后窗口在图上滑动，把所有局部分数平均起来。

这就是为什么 SSIM 通常比 PSNR 更接近人的视觉感受：人眼对结构、边缘、对比度更敏感，而不只是对像素差值敏感。

Part 1-任务三

```
(.venv) (base) PS C:\Users\ASUS\Desktop\Image-Deraining-Recruitment-Task-main> py
thon -c "import torch; print(torch.__version__, torch.cuda.is_available())"
>> pip list
>>
2.14.1+cpu False
Package           Version
----------------- ----------
colorama          0.4.6
filelock          4.2.4
fsspec            2026.9.0
ImageIO           2.38.1
Jinja2            3.1.6
lazy-loader       0.6
lpips             0.1.4
MarkupSafe        3.0.4
mpmath            1.3.0
networkx          3.7
numpy             2.5.4
packaging         26.3
pillow            13.0.0
pip               26.2.2
scikit-image      0.26.0
scipy             1.18.1
setuptools        84.2.1
sympy             1.14.0
tifffile          2026.10.15
torch             2.14.1
torchvision       0.29.1
tqdm              4.70.1
typing_extensions 4.16.0
(.venv) (base) PS C:\Users\ASUS\Desktop\Image-Deraining-Recruitment-Task-main>
```

```python
import torch.nn as nn

class BaselineNet(nn.Module):
    def __init__(self):
        super(BaselineNet, self).__init__()
        channels = 64
        self.body = nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding = 1, bias = False),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace = True),
            nn.Conv2d(channels, channels, 3, padding = 1, bias = False),
            nn.BatchNorm2d(channels),
        )
        self.relu = nn.ReLU(inplace = True)

    def forward(self, x):
        return self.relu(x + self.body(x))
```

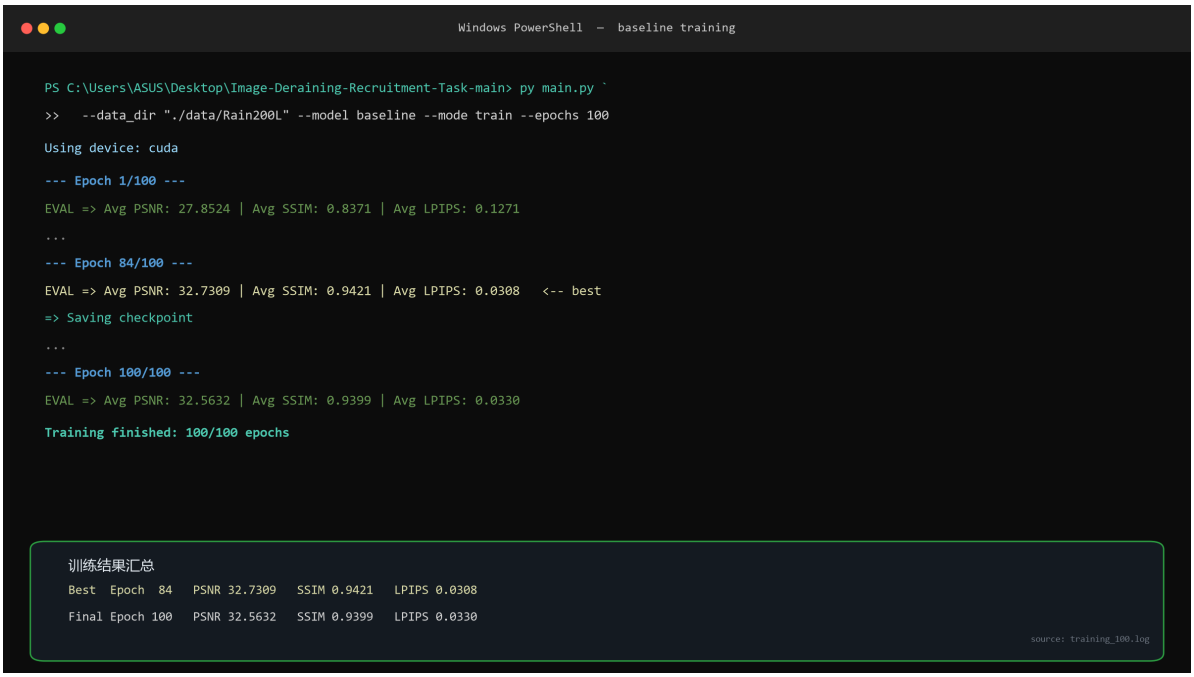
思考：

`3x3` 的卷积核感受野有限，在不改变分辨率（因为是残差学习，无雨图 = 原图 - 雨，所以必须保证分辨率相同）的情况下，只能不断加深网络层数，但感受野线性增长，这需要堆叠非常深的网络，会导致过拟合、梯度消失、网络退化。

通过观察我们发现，图片中的雨丝形状多为细长条，所以我们可以适当改变卷积核大小（比如改为`3x10`），并且在随层数增大的过程中卷积核大小也适当增大，一次在有限层数中获得尽可能大的感受野。

```
python main.py --data_dir "你的数据集地址" --model baseline --mode train --epochs 100
```

这个代码在终端中运行，`data_dir`中填写数据集地址即去雨训练用的图片，`model`选择模型，`mode`选择训练集或测试集，`epochs`表示训练轮数。





Part 2-任务一

```
parser.add_argument("--model", type=str, default="baseline",
                    choices=["baseline", "unet", "DerainNet", "Mymodel"],
                    help="Model to use")
```

baseline_net.py	2026/10/26 14:42	Python 源文件	1 KB
DerainNet.py	2026/10/26 14:42	Python 源文件	1 KB
Mymodel.py	2026/10/26 14:42	Python 源文件	1 KB
unet.py	2026/10/25 13:23	Python 源文件	2 KB

```
if args.model == "baseline":
    model = BaselineNet().to(device)
elif args.model == "Mymodel":
    model = BaselineNet().to(device)
elif args.model == "DerainNet":
    model = BaselineNet().to(device)
elif args.model == "unet":
    model = UNet().to(device)
```

Part 2-任务二

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class ResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.body = nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding = 1, bias = False),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace = True),
            nn.Conv2d(channels, channels, 3, padding = 1, bias = False),
            nn.BatchNorm2d(channels),
        )
        self.relu = nn.ReLU(inplace = True)

    def forward(self, x):
        return self.relu(x + self.body(x))


class DerainNet(nn.Module):
    def __init__(self, channels = 64):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, channels, 3, padding = 1, bias = False),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace = True),
        )
        self.branch_full = nn.Sequential(*[ResBlock(channels) for _ in
range(4)])
        self.branch_half = nn.Sequential(*[ResBlock(channels) for _ in
range(4)])
        self.branch_quarter = nn.Sequential(*[ResBlock(channels) for _ in
range(4)])
        self.pool = nn.MaxPool2d(kernel_size = 2, stride = 2)
```

```
        self.fuse = nn.Sequential(
            nn.Conv2d(3 * channels, channels, 3, padding = 1, bias = False),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace = True),
        )
        self.head = nn.Conv2d(channels, 3, 1)

    def forward(self, x):
        feat = self.stem(x)
        half_in = self.pool(feat)
        quarter_in = self.pool(half_in)

        branches = (self.branch_full, self.branch_half, self.branch_quarter)
        inputs = (feat, half_in, quarter_in)
        features = []
        for branch, inp in zip(branches, inputs):
            out = torch.relu(inp + branch(inp))
            features.append(
                F.interpolate(out, size = feat.shape[-2:], mode = "bilinear",
align_corners = False)
            )

        fused = self.fuse(torch.cat(features, dim = 1))
        rain = torch.relu(self.head(fused))
        return x - rain
```

stem做一次，把 RGB 的 3 个通道投影到 channel个特征通道，三个分支分别工作在 原图、1/2、1/4三种分辨率上，每个分支都是 4 个 ResBlock = 8 层 3×3 卷积，每个分支内部先做一次"处理前特征 + 处理后特征"的残差相加，再上采样回原图尺寸，三个分支在通道维拼接成 3channel 通道，用卷积融合，最后输出 3 通道雨层，最终`去雨图 = 输入 - 雨层`得到。

原因：

我们使用先下采样再上采样的方式可以用相对小的损失增大感受野，不再需要堆叠网络层数。同时我们也可以处理不同尺寸、疏密程度的雨丝。

下采样：

就是把特征图的空间尺寸变小，增大感受野。

例如：`128×128 → 64×64 → 32×32`

本题中我们使用池化操作来实现：`self.pool = nn.MaxPool2d(kernel_size=2, stride=2)`

上采样：

同理就是把特征图的空间尺寸变大。

例如：`32×32 → 64×64 → 128×128`

本题中我们使用双线性插值来实现：

```
F.interpolate(half, size=feat.shape[-2:], mode="bilinear", align_corners=False)
```

双线性插值的意思是：输出像素的值，由它周围最近的 4 个输入像素按距离加权平均得到，离哪个像素近，哪个像素的权重就大。

Concat：

concat就是拼接。

```
torch.cat([full, half, quarter], dim=1)
```

三个分支的形状都是 `[B, C, H, W]`，沿通道维 `dim=1` 拼接后变成：

```
[B, C, H, W] × 3  ──Concat(dim = 1)──→  [B, 3C, H, W]
```

Part 2-任务三

特征图中的L1：

LPIPS的优势：

原来的L1是将逐像素做差求和，找到像素点之间的差异，因此是局限的对'像素'优化。

而VGG模型中，因为特征图中的点不再是单一像素，而有着更大感受野，涵盖了局部甚至全局的信息，因此我们将其做差再求和就可以对'感知'进行优化。

训练网络的作用：

模型已经学会了基础的纹理细节，也可以做到识别物体，因此其特征图中的点涵盖了更多信息，它把"两张图像素是否接近"转换成"两张图在特征空间是否接近"。

```
import torch
import torch.nn as nn
from torchvision.models import vgg19, VGG19_Weights


class PerceptualLoss(nn.Module):
    def __init__(self):
        super(PerceptualLoss, self).__init__()
        self.vgg = vgg19(weights = VGG19_Weights.IMAGENET1K_V1).features[:35]
        self.vgg.eval()
        for param in self.vgg.parameters():
            param.requires_grad = False
        self.meanm = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
        self.std = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)
        self.l1 = nn.L1Loss()
    def forward(self, generated, ground_truth):
        generated = (generated - self.mean) / self.std
        ground_truth = (ground_truth - self.mean) / self.std
        gen_feat = self.vgg(generated)
        gt_feat = self.vgg(ground_truth)
        return self.l1(gen_feat, gt_feat)
```

向量：

在程序设计中我认为向量就是数组，而向量的维度就是数组的大小即特征向量中的特征数。

内积：

a，b两个n维向量。

两种表示方式：

$a \cdot b = a_1b_1 + a_2b_2 + \ldots + a_nb_n$

$a \cdot b = ||a|| \times ||b|| \times cos\theta$，其中$||a|| = \sqrt{({a_1}^2 + {a_2}^2 + \ldots + {a_n}^2)}$

在线性代数中：

矩阵乘法中的行与列以此相乘再求和，就是求内积。

内积的意义：内积衡量两个向量"方向有多一致、彼此有多少重叠"。内积把每一维的乘积加起来，如果两个向量在某一维**同号**，这一维贡献正数，如果**异号**，贡献负数，如果某一维有一个是 0，这一维没有贡献。可以体现相关性。

余弦相似度是把长度影响去掉后，只看方向的一致程度。

AdaptiveAvgPool2d：

自适应平均池化：不管输入特征图多大，都把它平均成你指定的输出尺寸。

自适应可以根据输入图像大小而调整池化窗口大小，从而保证输出尺寸不变。

easy contrastive loss：

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision.models import vgg19, VGG19_Weights

class Easy_contrastive_loss(nn.Module):
    def __init__(self):
        super(Easy_contrastive_loss, self).__init__()
        self.vgg = vgg19(weights = VGG19_Weights.IMAGENET1K_V1).features[:35]
        self.vgg.eval()
        for param in self.vgg.parameters():
            param.requires_grad = False
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
        self.std = torch.tensor([0.222, 0.224, 0.225]).view(1, 3, 1, 1)

    def encode(self, x):
        x = (x - self.mean) / self.std
        feat = self.vgg(x)
        feat = self.pool(feat)
        feat = feat.flatten(1)
        return F.normalize(feat, dim = 1)

    def forward(self, rainy, generated, ground_truth):
        rain_vec = self.encode(rainy)
        gt_vec = self.encode(ground_truth)
```

```
        gen_vec = self.encode(generated)
        cos_gt = F.cosine_embedding_loss(gen_vec, gt_vec, dim = 1)
        cos_rain = F.cosine_embedding_loss(gen_vec, rain_vec, dim = 1)
        loss = (1 - cos_gt + cos_rain).mean()
        return loss
```

main中添加：

```
    parser.add_argument("--use_contrastive_loss", action="store_true", help="Use easy contrastive loss")
    parser.add_argument("--lambda_contrastive", type=float, default=0.1, help="Weight for easy contrastive loss")
```

